PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
B. Tech. CSE - 5th Semester
Aug – Dec 2025

UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

PitchPal - "A platform to connect startups with investors"

Submitted by: Team Code:190-238

PES1UG23CS190	Dhruv Menon	5D
PES1UG23CS238	Harshini Dharniraj	5D

*Class of Prof. _**Raghu B. A.**_*
[Lab Faculty]

Table of Contents

INTRODUCTION

1. Problem Statement

The startup funding process lacks a unified system where founders can easily discover eligible investors, investors can evaluate startups based on domain preferences, and administrators can ensure safe and credible interactions. Communication between founders and investors often occurs across multiple platforms, making it difficult to track conversations, funding commitments, and investment progress.

Similarly, investor approvals and message moderation are typically manual, slow, and prone to inconsistencies. Without a centralized database, maintaining accurate records of funding rounds, domain interests, and pitch requests becomes unreliable.

PitchPal aims to overcome these limitations by designing a comprehensive relational database that:

- Stores detailed founder, investor, startup, and domain information
- Tracks investments and funding rounds accurately
- Maintains a secure messaging system between founders and investors
- Supports admin-level verification and moderation
- Provides intelligent investor matching based on startup domains
- Ensures consistency with triggers, stored procedures, and functions

Through this DBMS, PitchPal creates a seamless environment for structured, safe, and efficient investment interactions

2. Short Description

PitchPal is a digital platform that streamlines the interaction between founders and investors by providing structured startup listings, domain-based investor matching, secure communication channels, and transparent tracking of investments. The system includes three primary user groups Founders, Investors, and Admins each with clearly defined roles.The platform's database supports the lifecycle of a startup from creation, domain assignment, investor connections, funding rounds, to administrative oversight. Features such as investor approval, message moderation, triggers for dynamic funding updates, and recommendation-based investor matching make PitchPal a comprehensive solution for managing startup-investor relationships.

USER REQUIREMENT SPECIFICATION

Functional Requirements

Founder Module

- Register and create a profile
- Create and manage startup details
- Assign startup domains
- View potential investors based on domain match
- Initiate pitches to investors
- Exchange messages

Investor Module

- Register and create profile
- Define preferred investment domains
- Set investment range (min & max)
- View startups within preferred domains
- Receive pitch requests
- Communicate with founders

Admin Module

- Approve new investors
- Moderate messages
- Manage platform roles
- Review flagged interactions
- Access all system logs

System-Level Requirements

- Store and manage funding rounds with automatic updates
- Enforce constraints like investment range validation

E-R Model

Entities and Attributes

1. Founder

- founder_id (PK)
- name
- email
- password

2. Investor

- investor_id (PK)
- name
- email
- password
- funds
- min_investment
- max_investment

3. Domain

- domain_id (PK)
- d_name

4. Startup

(Weak relationship w.r.t Founder)

- startup_id (PK)
- founder_id (FK)
- domain_id (FK)
- name
- stage
- funding
- description

5. FundingRound

- funding_round_id (PK)
- startup_id (FK)
- investor_id (FK)
- amount
- date

6. PitchMatch

- pitch_match_id (PK)
- founder_id (FK)
- investor_id (FK)
- pitch_date
- status

7. Message

- m_id (PK)
- founder_id (FK)
- investor_id (FK)
- content
- sender_type
- timestamp

8. Admin

- admin_id (PK)
- name
- email
- role
- access_level

9. AdminInvestorApproval

- admin_id (FK)
- investor_id (FK)
- approval_date

10. MessageModeration

- admin_id (FK)
- m_id (FK)
- action
- action_date

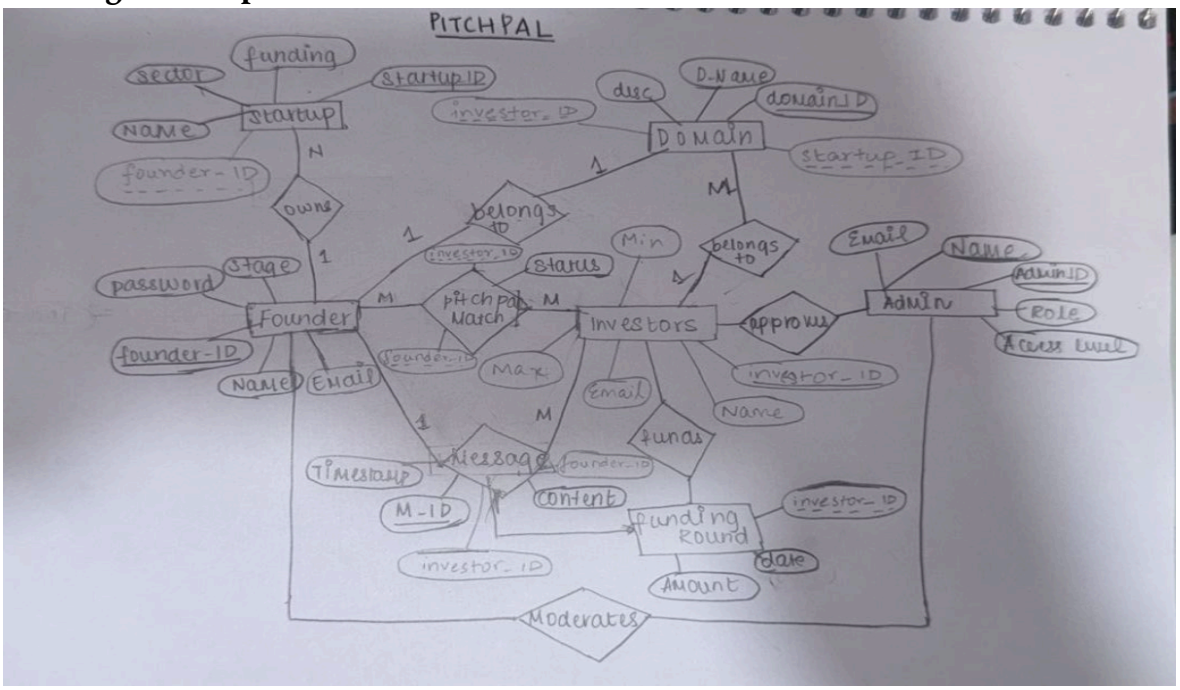
11. InvestorDomain

- investor_id (FK)
- domain_id (FK)

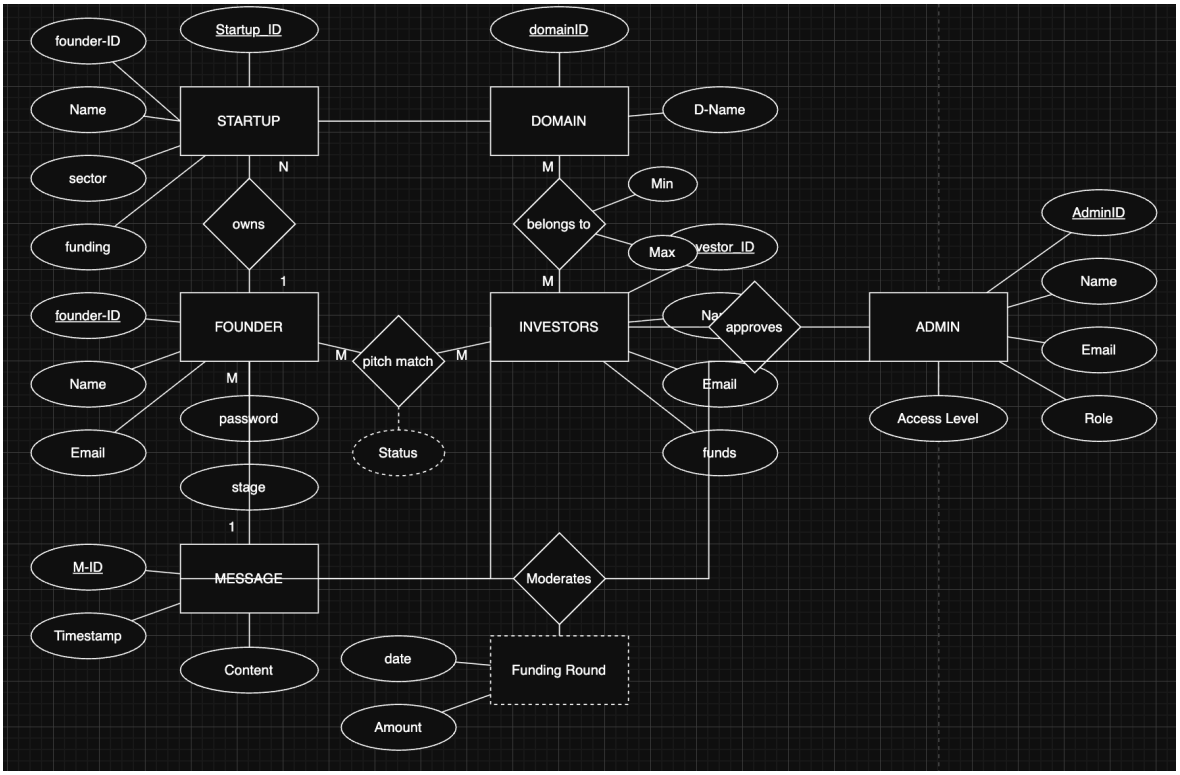
Weak Entities

- Startup (depends on Founder)
- PitchMatch (uniqueness defined by founder_id + investor_id)
- InvestorDomain (associative)
- AdminInvestorApproval (associative)
- MessageModeration (associative)

ER Diagram Snapshot



ERD from tool from draw.io



Relational Model

- **List of Relations**

Founder

Founder(founder_id PK, name, email, password)

Investor

Investor(investor_id PK, name, email, password, funds, min_investment, max_investment)

Domain

Domain(domain_id PK, d_name)

Startup

Startup(startup_id PK, founder_id FK, domain_id FK, name, stage, funding, description)

FundingRound

FundingRound(funding_round_id PK, startup_id FK, investor_id FK, amount, date)

PitchMatch

PitchMatch(pitch_match_id PK, founder_id FK, investor_id FK, pitch_date, status)

Message

Message(m_id PK, founder_id FK, investor_id FK, content, sender_type, timestamp)

InvestorDomain ***(Not an entity, but an associative relation)***

InvestorDomain(investor_id PK/FK, domain_id PK/FK)

Admin

Admin(admin_id PK, name, email, role, access_level)

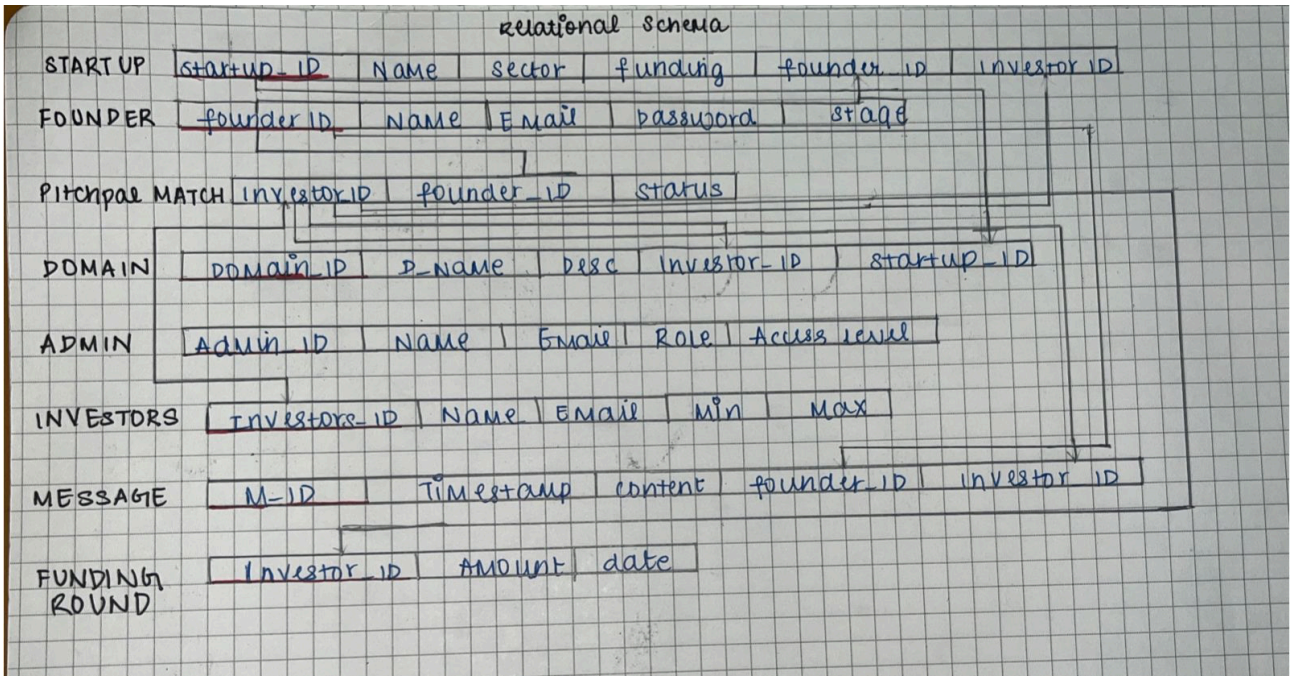
AdminInvestorApproval ***(Associative)***

AdminInvestorApproval(admin_id PK/FK, investor_id PK/FK, approval_date)

MessageModeration ***(Associative)***

MessageModeration(admin_id PK/FK, m_id PK/FK, action, action_date)

Schema Diagram



SQL DDL STATEMENTS

````
### 1. CREATE DATABASE
```sql
CREATE DATABASE IF NOT EXISTS PitchPalDB;
USE PitchPalDB;
```

### 2. DROP TABLE Statements
```sql
DROP TABLE IF EXISTS MessageModeration;
DROP TABLE IF EXISTS AdminInvestorApproval;
DROP TABLE IF EXISTS InvestorDomain;
DROP TABLE IF EXISTS Message;
DROP TABLE IF EXISTS PitchMatch;
DROP TABLE IF EXISTS FundingRound;
DROP TABLE IF EXISTS Startup;
DROP TABLE IF EXISTS Admin;
DROP TABLE IF EXISTS Investor;
DROP TABLE IF EXISTS Domain;
DROP TABLE IF EXISTS Founder;
```

### 3. CREATE TABLE Statements

#### Admin Table
```sql
CREATE TABLE Admin (
    admin_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    role VARCHAR(50),
    access_level VARCHAR(50)
);
```

#### Founder Table
```sql
````
````

```
CREATE TABLE Founder (
    founder_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL
);
```

Domain Table
```sql
CREATE TABLE Domain (
    domain_id INT PRIMARY KEY AUTO_INCREMENT,
    d_name VARCHAR(100) UNIQUE NOT NULL
);
```

Investor Table
```sql
CREATE TABLE Investor (
    investor_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    funds DECIMAL(15,2),
    min_investment DECIMAL(15,2),
    max_investment DECIMAL(15,2)
);
```

InvestorDomain Table
```sql
CREATE TABLE InvestorDomain (
    investor_id INT,
    domain_id INT,
    PRIMARY KEY (investor_id, domain_id),
    FOREIGN KEY (investor_id) REFERENCES Investor(investor_id) ON DELETE
CASCADE,
    FOREIGN KEY (domain_id) REFERENCES Domain(domain_id) ON DELETE CASCADE
);
```

```
```

Startup Table
```sql
CREATE TABLE Startup (
    startup_id INT PRIMARY KEY AUTO_INCREMENT,
    founder_id INT NOT NULL,
    domain_id INT,
    name VARCHAR(100) NOT NULL,
    stage VARCHAR(50),
    funding DECIMAL(15,2) DEFAULT 0.00,
    description TEXT,
    FOREIGN KEY (founder_id) REFERENCES Founder(founder_id) ON DELETE
CASCADE,
    FOREIGN KEY (domain_id) REFERENCES Domain(domain_id) ON DELETE SET NULL
);
```

FundingRound Table
```sql
CREATE TABLE FundingRound (
    funding_round_id INT PRIMARY KEY AUTO_INCREMENT,
    startup_id INT NOT NULL,
    investor_id INT NOT NULL,
    amount DECIMAL(15,2) NOT NULL,
    date DATE NOT NULL,
    FOREIGN KEY (startup_id) REFERENCES Startup(startup_id) ON DELETE
CASCADE,
    FOREIGN KEY (investor_id) REFERENCES Investor(investor_id) ON DELETE
CASCADE
);
```

PitchMatch Table
```sql
CREATE TABLE PitchMatch (
    pitch_match_id INT PRIMARY KEY AUTO_INCREMENT,
    founder_id INT NOT NULL,
    investor_id INT NOT NULL,
```

```
    pitch_date DATE DEFAULT (CURDATE()),
    status VARCHAR(20) DEFAULT 'Pending',
    FOREIGN KEY (founder_id) REFERENCES Founder(founder_id) ON DELETE
CASCADE,
    FOREIGN KEY (investor_id) REFERENCES Investor(investor_id) ON DELETE
CASCADE,
    UNIQUE (founder_id, investor_id)
);
```

Message Table
```sql
CREATE TABLE Message (
    m_id INT PRIMARY KEY AUTO_INCREMENT,
    founder_id INT NOT NULL,
    investor_id INT NOT NULL,
    content TEXT NOT NULL,
    sender_type ENUM('FOUNDER', 'INVESTOR') NOT NULL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (founder_id) REFERENCES Founder(founder_id) ON DELETE
CASCADE,
    FOREIGN KEY (investor_id) REFERENCES Investor(investor_id) ON DELETE
CASCADE
);
```

AdminInvestorApproval Table
```sql
CREATE TABLE AdminInvestorApproval (
    admin_id INT,
    investor_id INT,
    approval_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (admin_id, investor_id),
    FOREIGN KEY (admin_id) REFERENCES Admin(admin_id) ON DELETE CASCADE,
    FOREIGN KEY (investor_id) REFERENCES Investor(investor_id) ON DELETE
CASCADE
);
```

````
#### MessageModeration Table
```sql
CREATE TABLE MessageModeration (
    admin_id INT,
    m_id INT,
    action VARCHAR(100),
    action_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (admin_id, m_id),
    FOREIGN KEY (admin_id) REFERENCES Admin(admin_id) ON DELETE CASCADE,
    FOREIGN KEY (m_id) REFERENCES Message(m_id) ON DELETE CASCADE
);
```

### 4. CREATE FUNCTION Statements

#### Function 1: Check Investor Approval Status
```sql
DELIMITER $$

CREATE FUNCTION fn_CheckInvestorApprovalStatus(
    in_investor_id INT
)
RETURNS VARCHAR(20)
READS SQL DATA
BEGIN
    DECLARE approval_count INT DEFAULT 0;

    SELECT COUNT(*)
    INTO approval_count
    FROM AdminInvestorApproval
    WHERE investor_id = in_investor_id;

    IF approval_count > 0 THEN
        RETURN 'Approved';
    ELSE
        RETURN 'Pending Approval';
    END IF;
END$$
````
````

```
DELIMITER ;
```

Function 2: Get Founder's Startup Count
```sql
DELIMITER $$

CREATE FUNCTION fn_GetFounderStartupCount(
    in_founder_id INT
)
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE startup_count INT DEFAULT 0;

    SELECT COUNT(*)
    INTO startup_count
    FROM Startup
    WHERE founder_id = in_founder_id;

    RETURN startup_count;
END$$

DELIMITER ;
```

5. CREATE PROCEDURE Statements

Procedure 1: Create a New Pitch
```sql
DELIMITER $$

CREATE PROCEDURE sp_CreatePitch(
    IN in_founder_id INT,
    IN in_investor_id INT
)
BEGIN
    DECLARE existing_pitch INT DEFAULT 0;
```

```
    SELECT COUNT(*)
    INTO existing_pitch
    FROM PitchMatch
    WHERE founder_id = in_founder_id AND investor_id = in_investor_id;

    IF existing_pitch = 0 THEN
        INSERT INTO PitchMatch (founder_id, investor_id, status)
        VALUES (in_founder_id, in_investor_id, 'Pending');
        SELECT 'Pitch created successfully.' AS message;
    ELSE
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'A pitch between this founder and investor
already exists.';
    END IF;
END$$

DELIMITER ;
```

Procedure 2: Find Investor Matches for a Founder

```sql
DELIMITER $$

CREATE PROCEDURE sp_GetInvestorMatches(
    IN in_founder_id INT
)
BEGIN
    SELECT
        i.investor_id,
        i.name,
        i.email,
        d.d_name AS matching_domain
    FROM Investor i
    JOIN InvestorDomain idom ON i.investor_id = idom.investor_id
    JOIN Domain d ON idom.domain_id = d.domain_id
    WHERE
        idom.domain_id IN (
            SELECT DISTINCT domain_id
            FROM Startup
```

```
            WHERE founder_id = in_founder_id
        )
        AND i.investor_id NOT IN (
            SELECT investor_id
            FROM PitchMatch
            WHERE founder_id = in_founder_id
        )
        AND i.investor_id IN (
            SELECT investor_id
            FROM AdminInvestorApproval
        );
END$$

DELIMITER ;
```

6. CREATE TRIGGER Statements

Trigger 1: Update Total Startup Funding after FundingRound Insert

```sql
DELIMITER $$

CREATE TRIGGER trg_AfterInsertFundingRound
AFTER INSERT ON FundingRound
FOR EACH ROW
BEGIN
    UPDATE Startup
    SET funding = funding + NEW.amount
    WHERE startup_id = NEW.startup_id;
END$$

DELIMITER ;
```

Trigger 2: Check Investment Range before FundingRound Insert

```sql
DELIMITER $$

CREATE TRIGGER trg_BeforeInsertFundingRound_CheckRange
```

```
BEFORE INSERT ON FundingRound
FOR EACH ROW
BEGIN
    DECLARE min_inv DECIMAL(15,2);
    DECLARE max_inv DECIMAL(15,2);

    SELECT min_investment, max_investment
    INTO min_inv, max_inv
    FROM Investor
    WHERE investor_id = NEW.investor_id;

    IF NEW.amount < min_inv OR NEW.amount > max_inv THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Investment amount is outside the investor''s
specified range.';
    END IF;
END$$

DELIMITER ;
```

SQL DDL STATEMENTS

```
### 1. INSERT INTO Statements

#### Insert into Admin
```sql
INSERT INTO Admin (name, email, role, access_level) VALUES
('Super Admin', 'admin@pitchpal.com', 'System Administrator', 'Full'),
('Jane Doe', 'jane.doe@pitchpal.com', 'Content Moderator', 'Limited'),
('John Smith', 'john.s@pitchpal.com', 'Support Specialist', 'Limited'),
('Priya Singh', 'priya.s@pitchpal.com', 'Investor Relations', 'Full'),
('Amit Patel', 'amit.p@pitchpal.com', 'Compliance Officer', 'Full');
```

Insert into Founder
```sql
INSERT INTO Founder (name, email, password) VALUES
('Arjun Mehta', 'arjun@startup.com', 'hash_arjun123'),
('Sneha Rao', 'sneha@startup.com', 'hash_sneha456'),
('Kiran Sharma', 'kiran@startup.com', 'hash_kiran789'),
('Rohan Desai', 'rohan@startup.com', 'hash_rohan123'),
('Meera Krishnan', 'meera@startup.com', 'hash_meera456'),
('Vikram Singh', 'vikram@startup.com', 'hash_vikram789'),
('Anjali Iyer', 'anjali@startup.com', 'hash_anjali111');
```

Insert into Domain
```sql
INSERT INTO Domain (d_name) VALUES
('FinTech'), ('EdTech'), ('HealthTech'), ('SaaS'), ('AI/ML'),
('E-commerce'), ('AgriTech'), ('Logistics');
```

Insert into Investor
```

```sql
INSERT INTO Investor (name, email, password, funds, min_investment,
max_investment) VALUES
('Anil Kumar', 'anilvc@invest.com', 'hash_anilvc', 50000000.00, 100000.00,
5000000.00),
('Meera Iyer', 'meeraangels@invest.com', 'hash_meeraang', 75000000.00,
500000.00, 10000000.00),
('Vikram Reddy', 'vikramseed@invest.com', 'hash_vikramseed', 30000000.00,
50000.00, 1000000.00),
('Nisha Shah', 'nishavc@invest.com', 'hash_nishavc', 100000000.00,
1000000.00, 20000000.00),
('Sandeep Verma', 'sandeepai@invest.com', 'hash_sandeepml', 60000000.00,
250000.00, 8000000.00),
('Alok Jain', 'alok.j@invest.com', 'hash_alokjain', 40000000.00, 50000.00,
2000000.00);
```

Insert into InvestorDomain

```sql
INSERT INTO InvestorDomain (investor_id, domain_id) VALUES
(1, 1), (1, 4),
(2, 2),
(3, 3), (3, 5),
(4, 4), (4, 6),
(5, 5), (5, 8),
(6, 7), (6, 1),
(2, 5),
(4, 8);
```

Insert into Startup

```sql
INSERT INTO Startup (founder_id, domain_id, name, stage, funding,
description) VALUES
(1, 1, 'PaySwift', 'Seed', 500000.00, 'Mobile-first payment gateway for
small retailers'),
(2, 2, 'LearnSphere', 'Series A', 2000000.00, 'AI-powered adaptive learning
platform for students'),
```

```
(3, 3, 'MediTrack', 'Pre-Seed', 150000.00, 'IoT-enabled patient monitoring
for urban hospitals'),
(4, 6, 'FreshCart', 'Series B', 5000000.00, 'Subscription based e-commerce
for organic groceries'),
(5, 7, 'CropIntel', 'Seed', 400000.00, 'AI-driven crop monitoring for
precision agriculture'),
(6, 8, 'QuickHaul', 'Series A', 3000000.00, 'Platform for optimizing
last-mile delivery logistics'),
(7, 4, 'CodeDeploy', 'Seed', 750000.00, 'A SaaS platform for automated code
deployment and monitoring');
```

Insert into FundingRound

```sql
INSERT INTO FundingRound (startup_id, investor_id, amount, date) VALUES
(1, 1, 500000.00, '2024-02-12'),
(2, 2, 2000000.00, '2024-01-25'),
(3, 3, 150000.00, '2024-03-10'),
(4, 4, 5000000.00, '2023-11-05'),
(5, 6, 400000.00, '2024-04-18'),
(6, 5, 3000000.00, '2024-05-20'),
(7, 1, 750000.00, '2024-06-01');
```

Insert into PitchMatch

```sql
INSERT INTO PitchMatch (founder_id, investor_id, status) VALUES
(1, 1, 'Accepted'), (2, 2, 'Accepted'), (3, 3, 'Pending'),
(4, 4, 'Accepted'), (5, 6, 'Accepted'), (6, 5, 'Pending'),
(7, 1, 'Accepted'), (1, 4, 'Rejected'), (5, 1, 'Pending');
```

Insert into Message

```sql
INSERT INTO Message (founder_id, investor_id, content, sender_type) VALUES
(1, 1, 'Hello Anil, thank you for the seed funding for PaySwift.',
'FOUNDER'),
(1, 1, 'You are welcome, Arjun. Keep up the great work.', 'INVESTOR'),
```

```
(2, 2, 'Hi Meera, LearnSphere is preparing for its next funding round.',
'FOUNDER'),
(3, 3, 'Dear Vikram, attaching the MediTrack prototype video for your
review.', 'FOUNDER'),
(4, 4, 'Hi Nisha, FreshCart has hit our quarterly targets.', 'FOUNDER'),
(4, 4, 'Excellent news, Rohan. Let us schedule a follow-up call.',
'INVESTOR'),
(5, 6, 'Hello Alok, we believe CropIntel can revolutionize the AgriTech
space.', 'FOUNDER'),
(7, 1, 'Hi Anil, attaching our monthly report for CodeDeploy.', 'FOUNDER');
```

Insert into AdminInvestorApproval

```sql
INSERT INTO AdminInvestorApproval (admin_id, investor_id) VALUES
(1, 1), (1, 2), (1, 3), (4, 4), (4, 5), (5, 6);
```

Insert into MessageModeration

```sql
INSERT INTO MessageModeration (admin_id, m_id, action) VALUES
(2, 1, 'Flagged for review'),
(2, 2, 'Approved'),
(2, 3, 'Approved'),
(2, 4, 'Flagged for review'),
(3, 5, 'Approved'),
(2, 6, 'Approved'),
(3, 8, 'Approved');
```

2. UPDATE Statement (within Trigger)

Update in Trigger: trg_AfterInsertFundingRound

```sql
UPDATE Startup
SET funding = funding + NEW.amount
WHERE startup_id = NEW.startup_id;
```

```
### 3. SELECT Statements (within Functions/Procedures)

#### SELECT in Function: fn_CheckInvestorApprovalStatus
```sql
SELECT COUNT(*)
INTO approval_count
FROM AdminInvestorApproval
WHERE investor_id = in_investor_id;
```

SELECT in Function: fn_GetFounderStartupCount
```sql
SELECT COUNT(*)
INTO startup_count
FROM Startup
WHERE founder_id = in_founder_id;
```

SELECT in Procedure: sp_CreatePitch
```sql
SELECT COUNT(*)
INTO existing_pitch
FROM PitchMatch
WHERE founder_id = in_founder_id AND investor_id = in_investor_id;
```

SELECT in Procedure: sp_GetInvestorMatches
```sql
SELECT
    i.investor_id,
    i.name,
    i.email,
    d.d_name AS matching_domain
FROM Investor i
JOIN InvestorDomain idom ON i.investor_id = idom.investor_id
JOIN Domain d ON idom.domain_id = d.domain_id
WHERE
    idom.domain_id IN (
        SELECT DISTINCT domain_id
```
```

```
        FROM Startup
        WHERE founder_id = in_founder_id
    )
    AND i.investor_id NOT IN (
        SELECT investor_id
        FROM PitchMatch
        WHERE founder_id = in_founder_id
    )
    AND i.investor_id IN (
        SELECT investor_id
        FROM AdminInvestorApproval
    );
```

SELECT in Trigger: trg_BeforeInsertFundingRound_CheckRange

```sql
SELECT min_investment, max_investment
INTO min_inv, max_inv
FROM Investor
WHERE investor_id = NEW.investor_id;
```


mysql> show tables;
Tables_in_pitchpaldb
admin
admininvestorapproval
domain
founder
fundinground
investor
investordomain
message
messagemoderation
pitchmatch
startup

```
mysql> DESCRIBE Admin;
+--------------+--------------+------+-----+---------+----------------+
| Field        | Type         | Null | Key | Default | Extra          |
+--------------+--------------+------+-----+---------+----------------+
| admin_id     | int          | NO   | PRI | NULL    | auto_increment |
| name         | varchar(100) | NO   |     | NULL    |                |
| email        | varchar(100) | NO   | UNI | NULL    |                |
| role         | varchar(50)  | YES  |     | NULL    |                |
| access_level | varchar(50)  | YES  |     | NULL    |                |
+--------------+--------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)

mysql> -- Alternative: DESC Admin;
mysql>
mysql> -- Founder Table
mysql> DESCRIBE Founder;
+------------+--------------+------+-----+---------+----------------+
| Field      | Type         | Null | Key | Default | Extra          |
+------------+--------------+------+-----+---------+----------------+
| founder_id | int          | NO   | PRI | NULL    | auto_increment |
| name       | varchar(100) | NO   |     | NULL    |                |
| email      | varchar(100) | NO   | UNI | NULL    |                |
| password   | varchar(255) | NO   |     | NULL    |                |
+------------+--------------+------+-----+---------+----------------+
4 rows in set (0.00 sec)

mysql>
mysql> -- Domain Table
mysql> DESCRIBE Domain;
+-----------+--------------+------+-----+---------+----------------+
| Field     | Type         | Null | Key | Default | Extra          |
+-----------+--------------+------+-----+---------+----------------+
| domain_id | int          | NO   | PRI | NULL    | auto_increment |
| d_name    | varchar(100) | NO   | UNI | NULL    |                |
+-----------+--------------+------+-----+---------+----------------+
2 rows in set (0.00 sec)

mysql>
mysql> -- Investor Table
mysql> DESCRIBE Investor;
+----------------+---------------+------+-----+---------+----------------+
| Field          | Type          | Null | Key | Default | Extra          |
+----------------+---------------+------+-----+---------+----------------+
| investor_id    | int           | NO   | PRI | NULL    | auto_increment |
| name           | varchar(100)  | NO   |     | NULL    |                |
| email          | varchar(100)  | NO   | UNI | NULL    |                |
| password       | varchar(255)  | NO   |     | NULL    |                |
| funds          | decimal(15,2) | YES  |     | NULL    |                |
| min_investment | decimal(15,2) | YES  |     | NULL    |                |
| max_investment | decimal(15,2) | YES  |     | NULL    |                |
+----------------+---------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)
```

```
mysql> -- Startup Table
mysql> DESCRIBE Startup;
+-------------+---------------+------+-----+---------+----------------+
| Field       | Type          | Null | Key | Default | Extra          |
+-------------+---------------+------+-----+---------+----------------+
| startup_id  | int           | NO   | PRI | NULL    | auto_increment |
| founder_id  | int           | NO   | MUL | NULL    |                |
| domain_id   | int           | YES  | MUL | NULL    |                |
| name        | varchar(100)  | NO   |     | NULL    |                |
| stage       | varchar(50)   | YES  |     | NULL    |                |
| funding     | decimal(15,2) | YES  |     | 0.00    |                |
| description | text          | YES  |     | NULL    |                |
+-------------+---------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)

mysql>
mysql> -- FundingRound Table
mysql> DESCRIBE FundingRound;
+------------------+---------------+------+-----+---------+----------------+
| Field            | Type          | Null | Key | Default | Extra          |
+------------------+---------------+------+-----+---------+----------------+
| funding_round_id | int           | NO   | PRI | NULL    | auto_increment |
| startup_id       | int           | NO   | MUL | NULL    |                |
| investor_id      | int           | NO   | MUL | NULL    |                |
| amount           | decimal(15,2) | NO   |     | NULL    |                |
| date             | date          | NO   |     | NULL    |                |
+------------------+---------------+------+-----+---------+----------------+
5 rows in set (0.00 sec)

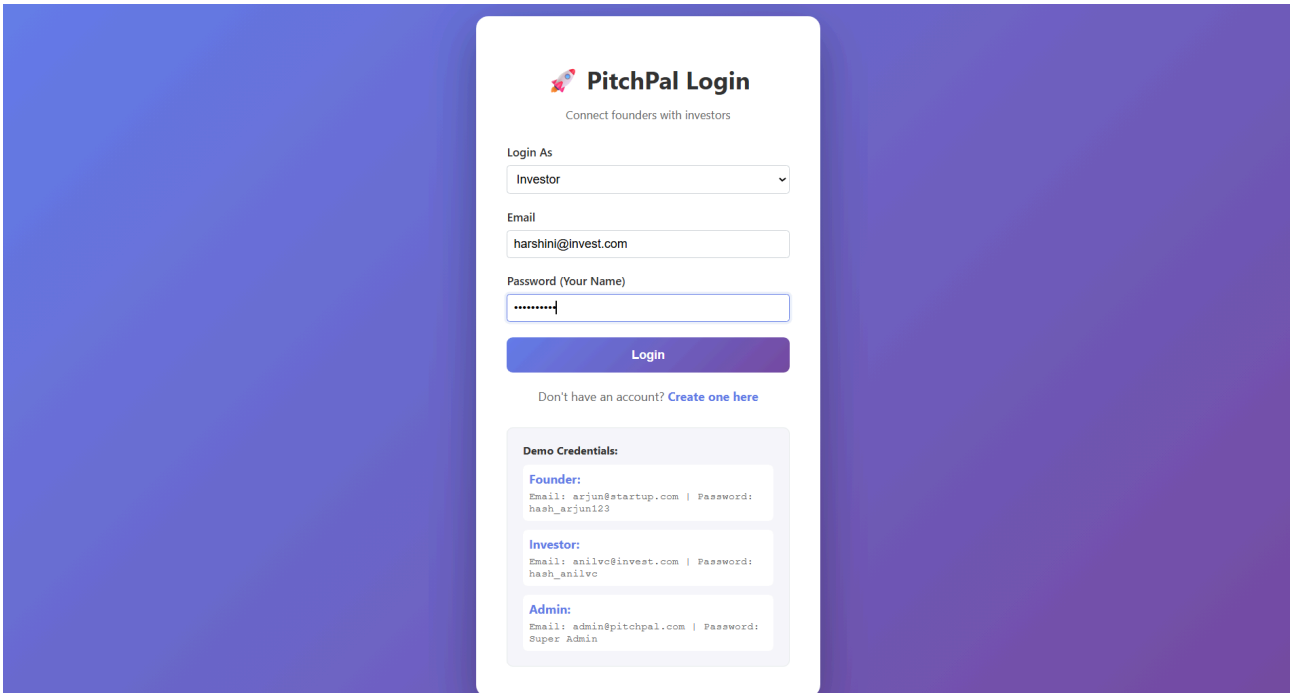
mysql>
mysql> -- PitchMatch Table
mysql> DESCRIBE PitchMatch;
+----------------+-------------+------+-----+-----------+-------------------+
| Field          | Type        | Null | Key | Default   | Extra             |
+----------------+-------------+------+-----+-----------+-------------------+
| pitch_match_id | int         | NO   | PRI | NULL      | auto_increment    |
| founder_id     | int         | NO   | MUL | NULL      |                   |
| investor_id    | int         | NO   | MUL | NULL      |                   |
| pitch_date     | date        | YES  |     | curdate() | DEFAULT_GENERATED |
| status         | varchar(20) | YES  |     | Pending   |                   |
+----------------+-------------+------+-----+-----------+-------------------+
5 rows in set (0.00 sec)

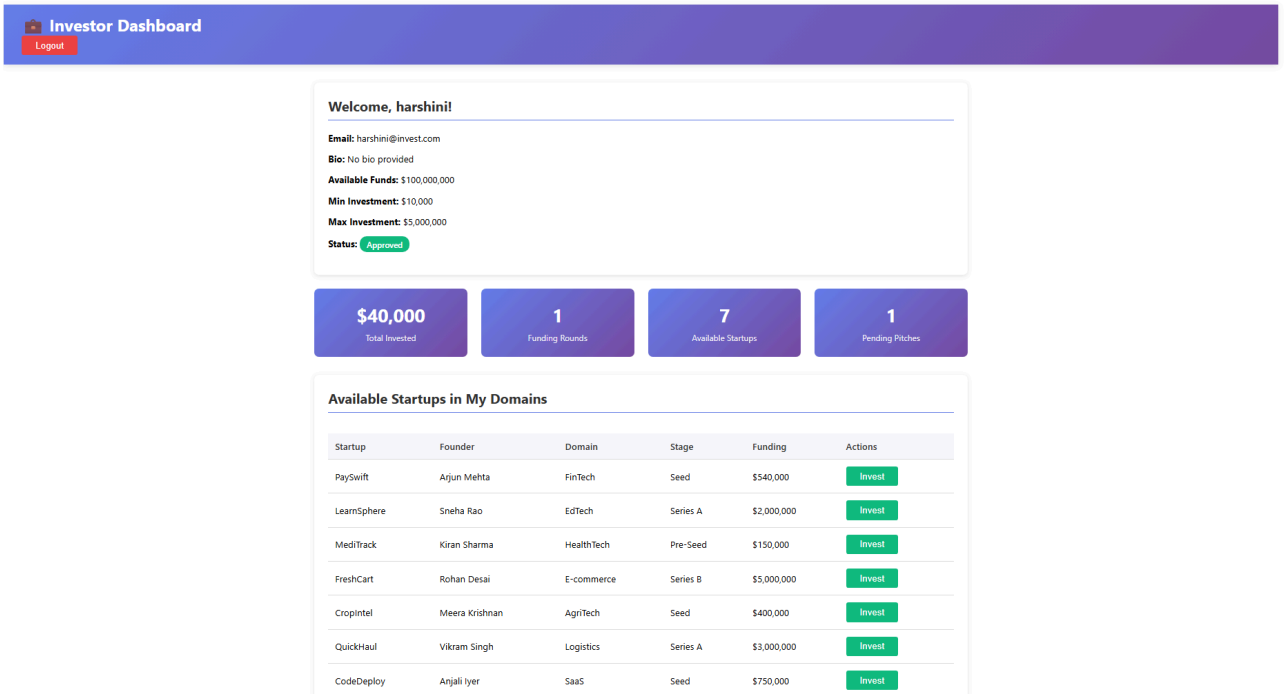
mysql>
mysql> -- Message Table
mysql> DESCRIBE Message;
+-------------+-------------------------+------+-----+-------------------+-------------------+
| Field       | Type                    | Null | Key | Default           | Extra             |
+-------------+-------------------------+------+-----+-------------------+-------------------+
| m_id        | int                     | NO   | PRI | NULL              | auto_increment    |
| founder_id  | int                     | NO   | MUL | NULL              |                   |
| investor_id | int                     | NO   | MUL | NULL              |                   |
| content     | text                    | NO   |     | NULL              |                   |
| sender_type | enum('FOUNDER','INVESTOR') | NO   |     | NULL              |                   |
| timestamp   | timestamp               | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-------------+-------------------------+------+-----+-------------------+-------------------+
6 rows in set (0.00 sec)

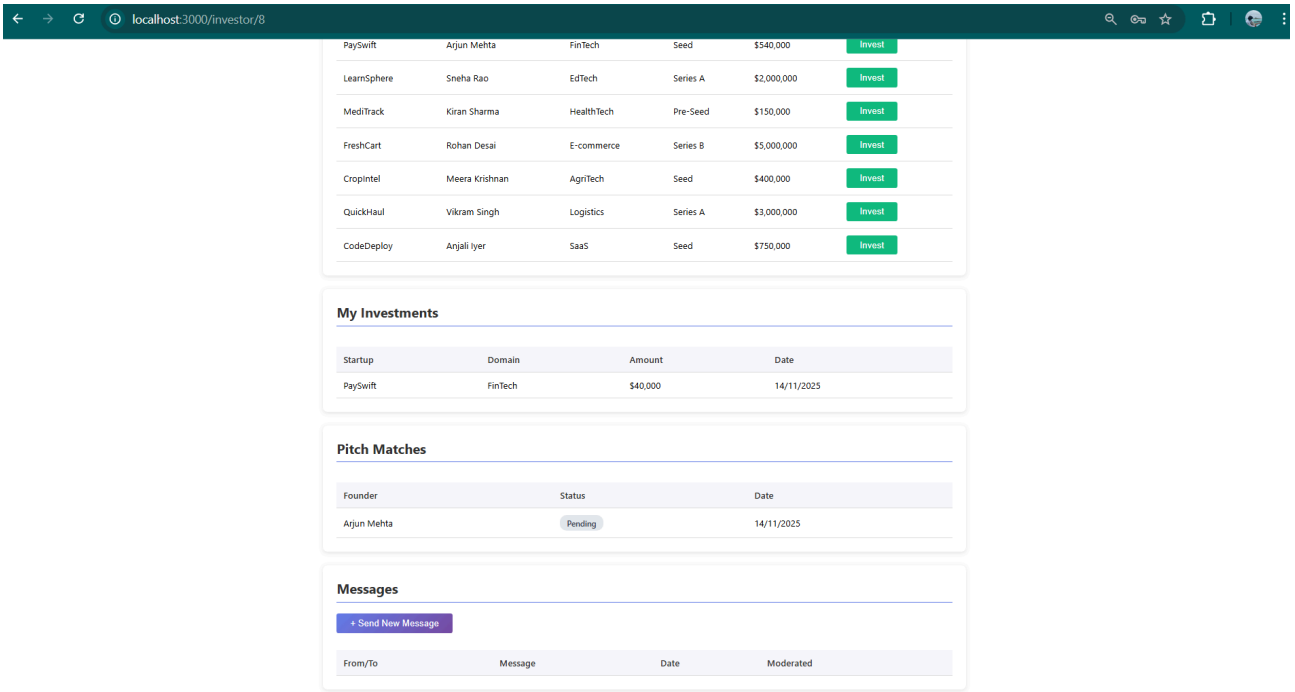
mysql>
mysql> -- AdminInvestorApproval Table
mysql> DESCRIBE AdminInvestorApproval;
+---------------+-----------+------+-----+-------------------+-------------------+
| Field         | Type      | Null | Key | Default           | Extra             |
+---------------+-----------+------+-----+-------------------+-------------------+
| admin_id      | int       | NO   | PRI | NULL              |                   |
| investor_id   | int       | NO   | PRI | NULL              |                   |
| approval_date | timestamp | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+---------------+-----------+------+-----+-------------------+-------------------+
3 rows in set (0.00 sec)

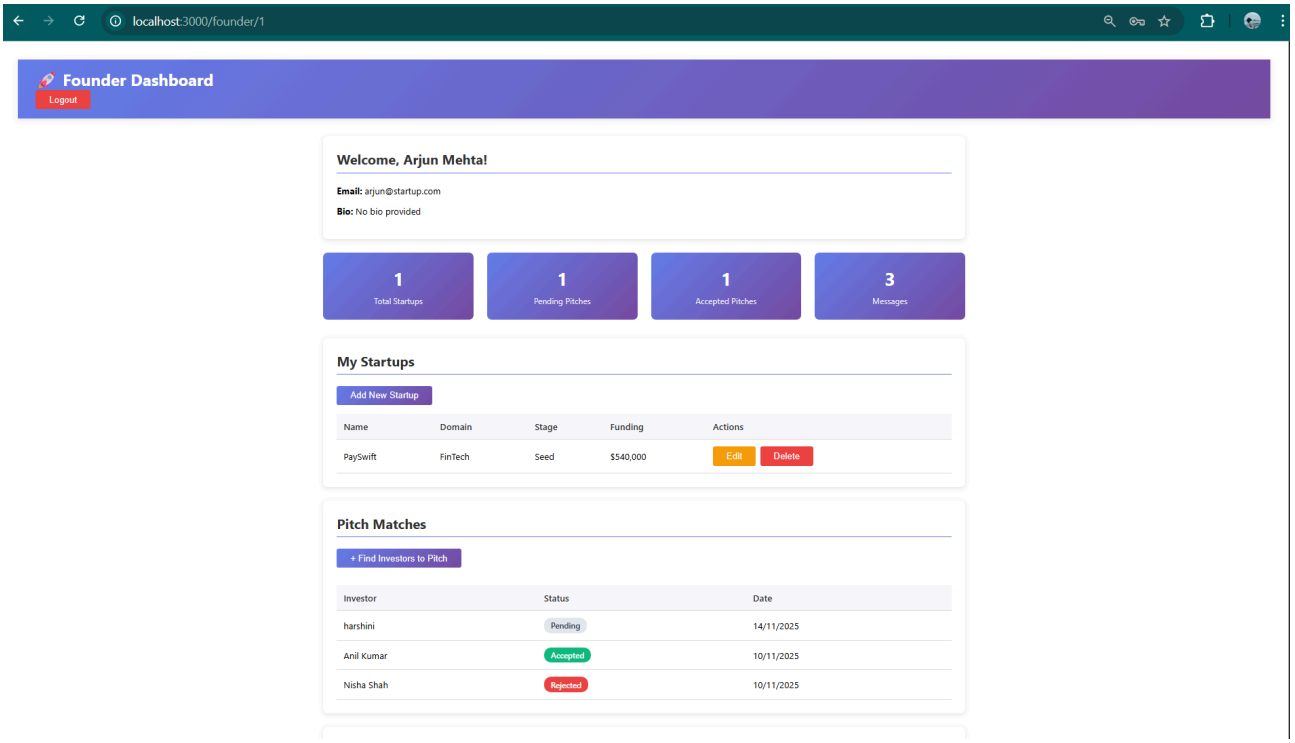
mysql>
mysql> -- MessageModeration Table
mysql> DESCRIBE MessageModeration;
```

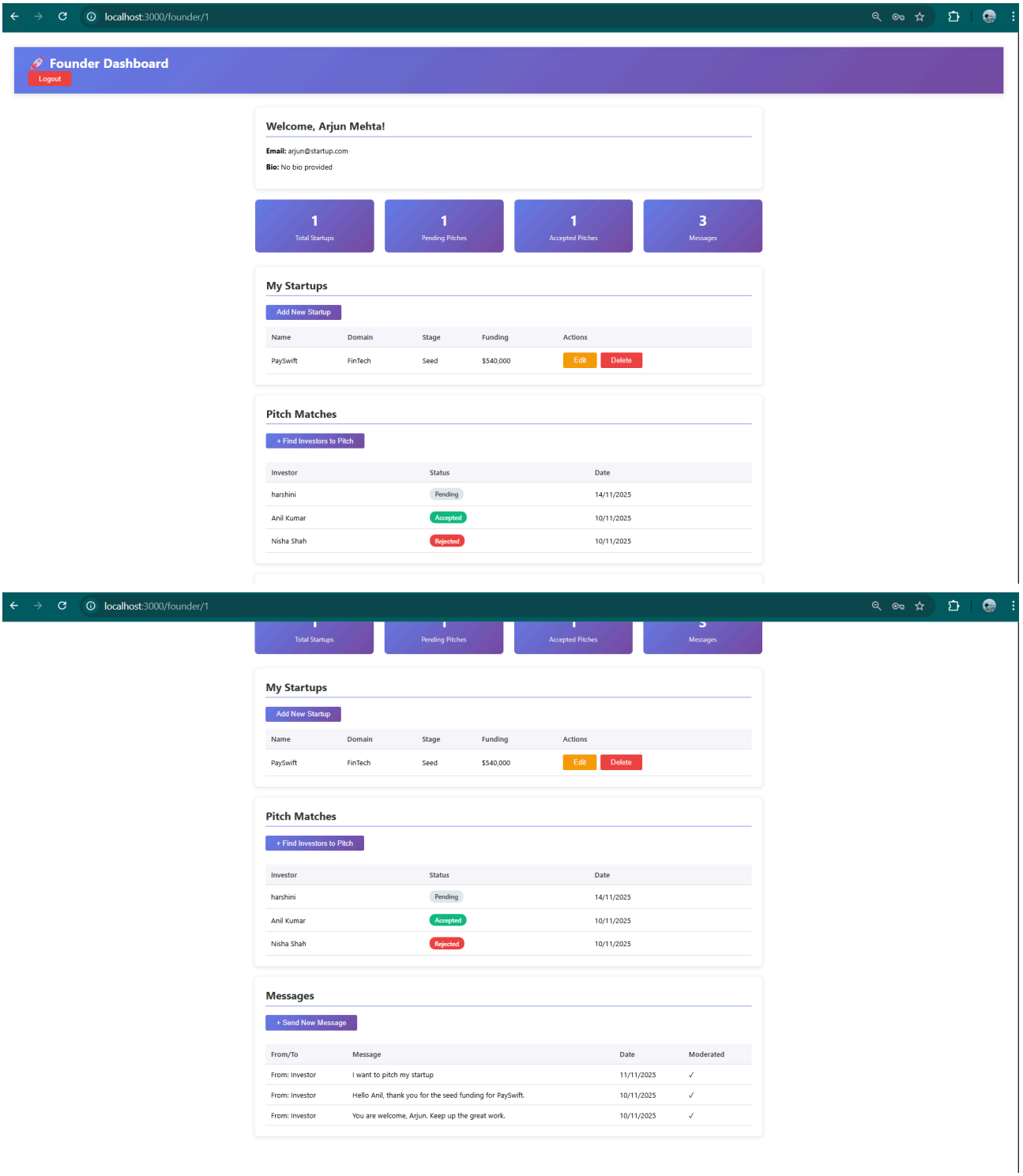
APPLICATIONS FRONT END SCREENSHOTS

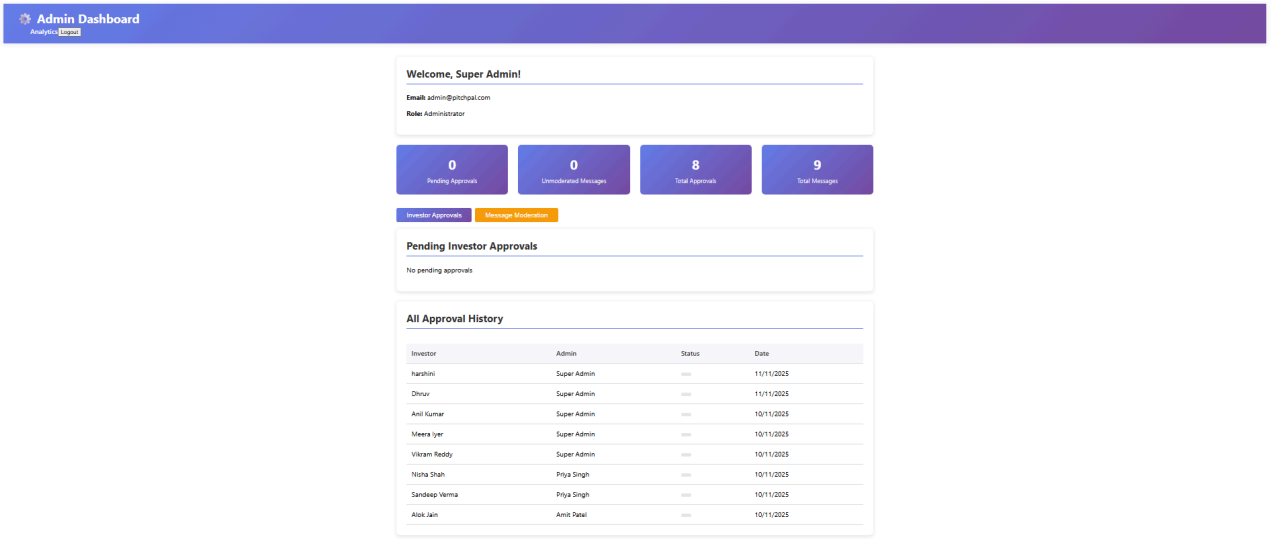


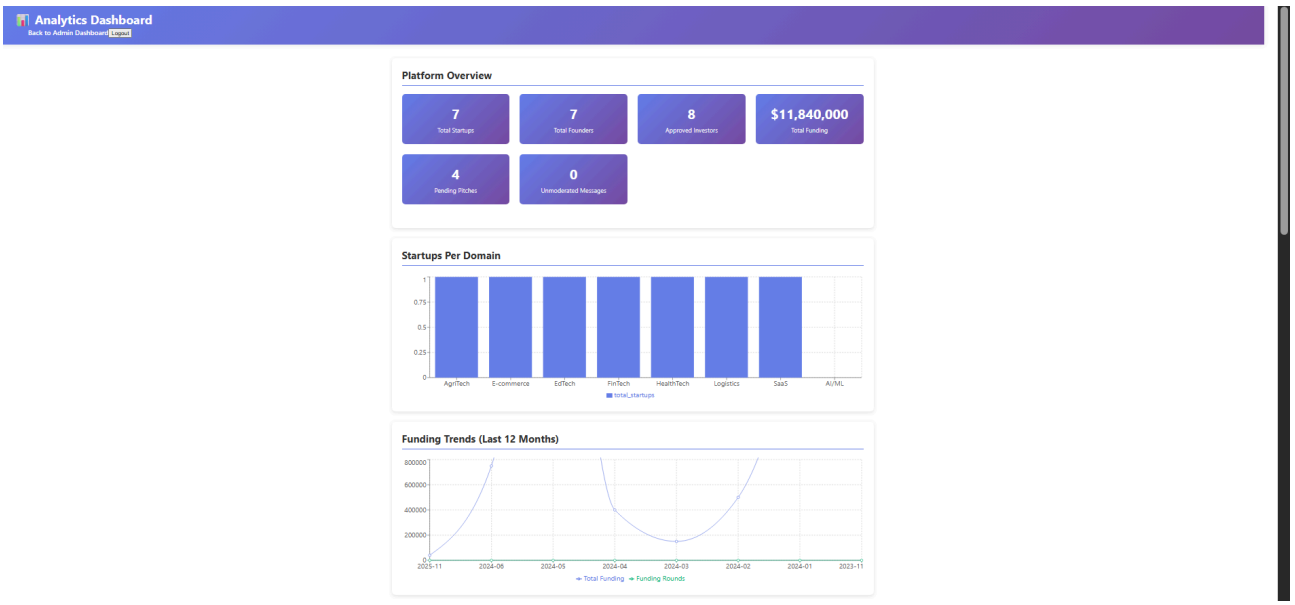


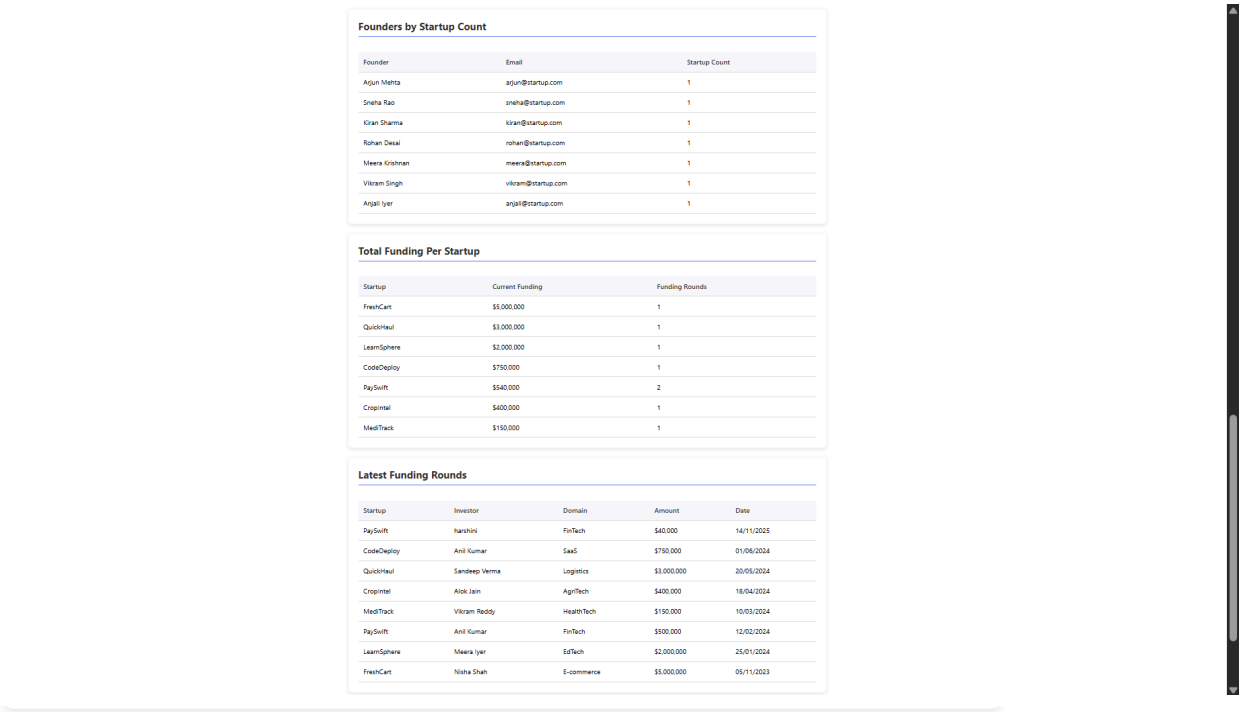

localhost:3000/investor/8
PaySwift Arjun Mehta FinTech Seed $540,000 Invest
LearnSphere Sneha Rao EdTech Series A $2,000,000 Invest
MediTrack Kiran Sharma HealthTech Pre-Seed $150,000 Invest
FreshCart Rohan Desai E-commerce Series B $5,000,000 Invest
CropIntel Meera Krishnan AgriTech Seed $400,000 Invest
QuickHaul Vikram Singh Logistics Series A $3,000,000 Invest
CodeDeploy Anjali Iyer SaaS Seed $750,000 Invest
My Investments
Startup Domain Amount Date
PaySwift FinTech $40,000 14/11/2025
Pitch Matches
Founder Status Date
Arjun Mehta Pending 14/11/2025
Messages
+ Send New Message
From/To Message Date Moderated


localhost:3000/founder/1
Founder Dashboard
Logout
Welcome, Arjun Mehta!
Email: arjun@startup.com
Bio: No bio provided
1 Total Startups
1 Pending Pitches
1 Accepted Pitches
3 Messages
My Startups
Add New Startup
Name Domain Stage Funding Actions
PaySwift FinTech Seed $540,000 Edit Delete
Pitch Matches
+ Find Investors to Pitch
Investor Status Date
harshini Pending 14/11/2025
Anil Kumar Accepted 10/11/2025
Nisha Shah Rejected 10/11/2025


localhost:3000/founder/1
Founder Dashboard
Logout
Welcome, Arjun Mehta!
Email: arjun@startup.com
Bio: No bio provided
1
Total Startups
1
Pending Pitches
1
Accepted Pitches
3
Messages
My Startups
Add New Startup
Name
Domain
Stage
Funding
Actions
PaySwift
FinTech
Seed
$540,000
Edit
Delete
Pitch Matches
+ Find Investors to Pitch
Investor
Status
Date
harshini
Pending
14/11/2025
Anil Kumar
Accepted
10/11/2025
Nisha Shah
Rejected
10/11/2025
Messages
+ Send New Message
From/To
Message
Date
Moderated
From: Investor
I want to pitch my startup
11/11/2025
From: Investor
Hello Anil, thank you for the seed funding for PaySwift.
10/11/2025
From: Investor
You are welcome, Arjun. Keep up the great work.
10/11/2025


Admin Dashboard
Analytics Logout
Welcome, Super Admin!
Email: admin@pitchpal.com
Role: Administrator
0
Pending Approvals
0
Unmoderated Messages
8
Total Approvals
9
Total Messages
Investor Approvals
Message Moderation
Pending Investor Approvals
No pending approvals
All Approval History
Investor
Admin
Status
Date
harshini
Super Admin
11/11/2025
Dhruv
Super Admin
11/11/2025
Anil Kumar
Super Admin
10/11/2025
Meera Iyer
Super Admin
10/11/2025
Vikram Reddy
Super Admin
10/11/2025
Nisha Shah
Priya Singh
10/11/2025
Sandeep Verma
Priya Singh
10/11/2025
Alok Jain
Amit Patel
10/11/2025


Analytics Dashboard
Back to Admin Dashboard Logout
Platform Overview
7
Total Startups
7
Total Founders
8
Approved Investors
$11,840,000
Total Funding
4
Pending Pitches
0
Unmoderated Messages
Startups Per Domain
AgriTech
E-commerce
EdTech
FinTech
HealthTech
Logistics
SaaS
AI/ML
total_startups
Funding Trends (Last 12 Months)
Total Funding
Funding Rounds



Founders by Startup Count

Founder	Email	Startup Count
Arjun Mehta	arjun@startup.com	1
Sneha Rao	sneha@startup.com	1
Kiran Sharma	kiran@startup.com	1
Rohan Desai	rohan@startup.com	1
Meera Krishnan	meera@startup.com	1
Vikram Singh	vikram@startup.com	1
Anjali Iyer	anjali@startup.com	1

Total Funding Per Startup

Startup	Current Funding	Funding Rounds
FreshCart	$5,000,000	1
QuickHaul	$3,000,000	1
LearnSphere	$2,000,000	1
CodeDeploy	$750,000	1
PaySwift	$540,000	2
CropIntel	$400,000	1
MediTrack	$150,000	1

Latest Funding Rounds

Startup	Investor	Domain	Amount	Date
PaySwift	harshini	FinTech	$40,000	14/11/2025
CodeDeploy	Anil Kumar	SaaS	$750,000	01/08/2024
QuickHaul	Sandeep Verma	Logistics	$3,000,000	20/05/2024
CropIntel	Alok Jain	AgriTech	$400,000	18/04/2024
MediTrack	Vikram Reddy	HealthTech	$150,000	10/03/2024
PaySwift	Anil Kumar	FinTech	$500,000	12/02/2024
LearnSphere	Meera Iyer	EdTech	$2,000,000	25/01/2024
FreshCart	Nisha Shah	E-commerce	$5,000,000	05/11/2023

Database Report Queries

Special query types: JOIN, AGGREGATE, and NESTED (Subquery)

Hide Reports ▲

JOIN Query: Startups with Founder & Domain Details

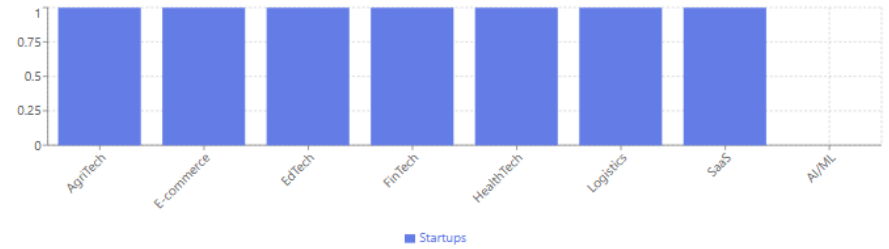
Query Type: Multi-table JOIN
Description: Lists all startups with their founder and domain information using INNER JOIN

Startup	Founder	Email	Domain	Stage	Funding
CodeDeploy	Anjali Iyer	anjali@startup.com	SaaS	Seed	$750,000
CropIntel	Meera Krishnan	meera@startup.com	AgriTech	Seed	$400,000
FreshCart	Rohan Desai	rohan@startup.com	E-commerce	Series B	$5,000,000
LearnSphere	Sneha Rao	sneha@startup.com	EdTech	Series A	$2,000,000
MediTrack	Kiran Sharma	kiran@startup.com	HealthTech	Pre-Seed	$150,000
PaySwift	Arjun Mehta	arjun@startup.com	FinTech	Seed	$540,000
QuickHaul	Vikram Singh	vikram@startup.com	Logistics	Series A	$3,000,000

```
SQL Query:
SELECT s.name, f.name, d.d_name
FROM Startup s
JOIN Founder f ON s.founder_id = f.founder_id
JOIN Domain d ON s.domain_id = d.domain_id
```

📊 AGGREGATE Query: Startup Count Per Domain

Query Type: GROUP BY with COUNT aggregate
Description: Counts number of startups in each domain using GROUP BY and aggregate function



Domain	Startup Count
AgriTech	1
E-commerce	1
EdTech	1
FinTech	1
HealthTech	1
Logistics	1
SaaS	1
AI/ML	0

```
SQL Query:
SELECT d.d_name, COUNT(s.startup_id) AS startup_count
FROM Domain d
LEFT JOIN Startup s ON d.domain_id = s.domain_id
GROUP BY d.d_name
ORDER BY startup_count DESC
```

🔍 NESTED (Subquery) Query: Investors by Domain

Query Type: Subquery with IN clause and nested SELECT
Description: Finds all investors interested in a specific domain using nested subqueries

Select Domain: FinTech

Investor Name	Email	Available Funds	Min Investment	Max Investment
Alok Jain	alok.j@invest.com	$40,000,000	$50,000	$2,000,000
Anil Kumar	anilvc@invest.com	$50,000,000	$100,000	$5,000,000
harshini	harshini@invest.com	$100,000,000	$10,000	$5,000,000

```
SQL Query:
SELECT name, email, funds
FROM Investor
WHERE investor_id IN (
  SELECT investor_id FROM InvestorDomain
  WHERE domain_id = (
    SELECT domain_id FROM Domain WHERE d_name = 'FinTech'
  )
)
```

CONCLUSION

The PitchPal database system successfully demonstrates how a well-designed relational DBMS can support the complete lifecycle of interactions within a real-world startup funding platform. By modelling founders, investors, admins, domains, funding rounds, pitch interactions, and moderated communication, the system provides a structured foundation for an otherwise fragmented and unpredictable investment ecosystem. Through carefully defined entities, associative relationships, and referential integrity constraints, PitchPal ensures that all data whether related to users, startups, funding activities, or messages is consistently stored, efficiently retrieved, and securely managed.

One of the core strengths of this DBMS is the logical mapping between real-life operational workflows and database-level mechanisms. Features such as investor–domain preferences, founder–startup ownership, pitch matching, investor approvals, and message moderation are captured in a way that preserves the integrity of the platform while enabling smooth interactions. The inclusion of triggers, stored functions, and stored procedures enhances the intelligence of the system by automating critical processes such as funding updates, investment range validation, pitch creation checks, and investor matching based on startup domains. These elements collectively ensure that the system is not only functionally rich but also reliable, scalable, and resistant to invalid operations.

The design enables administrators to maintain transparency and trust by verifying investors, moderating communication, and tracking sensitive interactions. Investors benefit from clearly defined preferences, automated compatibility matching, and controlled communication channels. Founders are empowered with a robust system for showcasing their startups, tracking pitches, and receiving structured feedback. This collaborative structure forms the backbone of PitchPal, transforming what is typically an unorganized process into a seamless, data-driven framework.

Overall, the PitchPal DBMS is a comprehensive solution that blends real-world requirements with academic principles of database design. It aligns accurately with normalization standards, uses meaningful constraints, and leverages SQL capabilities to support automation, data integrity, and system efficiency. The project demonstrates how a thoughtfully constructed database can become the central pillar of a scalable platform capable of managing significant data growth, supporting advanced business logic, and maintaining reliable interactions between all stakeholders. This DBMS lays a strong foundation for future platform development, such as recommendation systems, investor risk profiling, startup performance analytics, and deeper AI-assisted matchmaking, making PitchPal a powerful step toward modernizing the startup investor ecosystem.

.

Tools Used

- MySQL
- Workbench
- Draw.io for ER diagrams
- Python
- VS Code

REFERENCES

1. **PitchPal Project Details. (2025).** ***Internal Project Specification Document*****.** **Available in project submission: PitchPal_Project_Details.pdf.**

2. **Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019).** ***Database System Concepts*** **(7th ed.). McGraw-Hill Education.**

3. **Elmasri, R., & Navathe, S. (2015).** ***Fundamentals of Database Systems*** **(7th ed.). Pearson.**

4. **MySQL Documentation. (2024).** ***Triggers, Stored Procedures, Functions, Constraints*****.**

5. **MySQL Workbench User Manual. (2024). Oracle Corporation.**

6. **Draw.io (Diagrams.net). (2024).** ***ER Diagram Creation Tool*****.**

7. **W3Schools. (2024).** ***SQL Syntax and Functions Reference*****.**

8. **GeeksforGeeks. (2024).** ***Database Management Systems Tutorials*****.**